

GreenLLM: Disaggregating Large Language Model Serving on Heterogeneous GPUs for Lower Carbon Emissions

Tianyao Shi^{1,*}, Yanran Wu^{1,*}, Sihang Liu², and Yi Ding¹

¹Purdue University

²University of Waterloo

Abstract

LLMs have been widely adopted across many real-world applications. However, their widespread use comes with significant environmental costs due to their high computational intensity and resource demands. Specifically, this has driven the development of new generations of high-performing GPUs, exacerbating the problem of electronic waste and accelerating the premature disposal of devices.

To address this problem, this paper focuses on reducing the carbon emissions of LLM serving by reusing older, low-performing GPUs. We present GreenLLM, an SLO-aware LLM serving framework designed to minimize carbon emissions by reusing older GPUs. GreenLLM builds on two identified use cases that disaggregate specific computations onto older GPUs, reducing carbon emissions while meeting performance goals. To deepen our understanding of the potential carbon savings from disaggregation, we also provide a theoretical analysis of its relationship with carbon intensity and GPU lifetime. Our evaluations show that GreenLLM reduces carbon emissions by up to 40.6% compared to running standard LLM serving on new GPU only, meeting latency SLOs for over 90% of requests across various applications, latency requirements, carbon intensities, and GPU lifetimes.

1 Introduction

Large language models (LLMs) have been widely adopted in various real-world applications for their ability to perform complex natural language processing tasks [28, 39, 44]. Despite their widespread use, serving LLMs causes significant environmental impact in terms of carbon emissions¹. For example, processing a single ChatGPT prompt generates over 4 grams of CO₂eq [47], which is more than 20 times the *operational carbon emission* of a web search query [15].

Serving LLMs relies on high-performing but power-hungry GPUs. The rising demand for computation has driven the

development of new generations of high-performing GPUs, whose manufacturing processes contribute significantly to environmental impact in terms of *embodied carbon emissions* [19, 48]. Meanwhile, older GPUs become increasingly overlooked and inadequate for running the latest LLM serving systems, further exacerbating the growing problem of electronic waste (e-waste) [7, 11]. Recent studies estimate that e-waste from generative artificial intelligence (AI) could reach 1.2 to 5.0 million tons between 2020 and 2030 [46], with rapid advances in AI that accelerate obsolescence and premature disposal of devices [1, 42].

In this paper, we tackle the challenge of reducing the total carbon emissions of LLM serving by reusing old, low-performing GPUs. Since embodied carbon emissions are amortized over a device’s lifetime, extending hardware lifetimes by reusing older GPUs will effectively reduce these emissions [18]. To this end, we explore GPU heterogeneity to enable the reuse of older GPUs for serving LLMs, while meeting performance goals. Specifically, we investigate strategies for disaggregating a portion of the computation of LLM serving to older GPUs to reduce total carbon emissions.

We first explore allocating the prefill and decoding phases of LLM serving onto different types of GPUs. LLM serving operates in two phases—prefill and decoding (as detailed in §2.1). The prefill phase processes the prompt, stores the context in the key-value (KV) cache, and is compute-bound, while the decoding phase generates tokens autoregressively and is memory-bound. Prior work has either separated the prefill and decoding phases on homogeneous GPUs to eliminate interference and improve performance [22, 50], or heterogeneous GPUs to reduce power and costs [16, 34]. In contrast, we minimize carbon emissions by using heterogeneous GPUs, especially old, low-performing GPUs. We allocate the prefill phase to newer, more advanced GPU for faster prompt processing, while assigning the decoding phase to older, less advanced GPU to reduce embodied carbon emissions. We call this approach **Disg-Pref-Decode**, and our experiments show that it can effectively reduce total carbon emissions while meeting latency service level objectives (SLOs).

^{*}These authors contributed equally.

¹Detailed definitions of various carbon emissions are provided in §2.3.

While Disg-Pref-Decode demonstrates a feasible use case to reduce carbon emissions for LLM serving, it has a key limitation: the high network bandwidth requirements due to the large KV cache transfer between GPUs. For instance, disaggregating the prefill and decoding phases of a Llama-7B model on A100 and T4 GPUs requires bandwidth over 10Gbps even when the request rate of 1 prompt/s (see Figure 4). In practice, the carbon reduction benefits of Disg-Pref-Decode can diminish as the KV cache size increases, particularly because the network devices connecting heterogeneous GPUs can have slower data transfer speeds. This overhead can offset the gains achieved by Disg-Pref-Decode when high network bandwidth is not available.

To address this limitation, we explore a second disaggregation use case based on speculative decoding for systems without high-bandwidth interconnect, while still reducing total carbon emissions. Speculative decoding accelerates LLM serving by running two LLM models concurrently [24, 30]: a larger, slower model and a smaller, faster model. Instead of generating tokens sequentially as in standard LLMs, the smaller model generates a set of tokens, and the larger model verifies them in parallel. This approach reduces the computational load on the larger model by offloading initial predictions to the smaller model, allowing it to focus primarily on validation. Building on this concept, we introduce a disaggregated version of speculative decoding (**Disg-Spec-Decode**) using heterogeneous GPUs. In Disg-Spec-Decode, the larger model is assigned to the newer GPU, while the smaller model runs on the older GPU. Unlike Disg-Pref-Decode, Disg-Spec-Decode has a low network bandwidth requirement since it only transfers speculative tokens and the corresponding probability distribution between GPUs.

Building on the two use cases we identified, we design and implement GreenLLM, an SLO-aware LLM serving system that minimizes total carbon emissions by leveraging heterogeneous GPUs. GreenLLM includes three components: a disaggregated system supporting various LLM serving configurations such as Disg-Pref-Decode and Disg-Spec-Decode; a profiler measuring performance and energy consumption across various request sizes and rates; and an SLO-aware scheduler that searches for the optimal configuration for minimal carbon emissions while meeting latency SLOs.

To further understand disaggregation’s carbon savings, we theoretically compare standalone (new GPU only) and disaggregated (new and old GPUs) setups. We address three questions: 1) When does disaggregation yield greater carbon savings? 2) How does carbon intensity influence carbon savings? 3) How does GPU lifetime influence carbon savings?

We evaluate GreenLLM on three real-world LLM applications: chatbots, programming assistants, and document summarization. Compared to running standard LLM serving on new GPU only, GreenLLM reduces carbon emissions by up to 40.6% while meeting latency SLOs. Additionally, GreenLLM achieves up to 27.9% carbon savings across regions

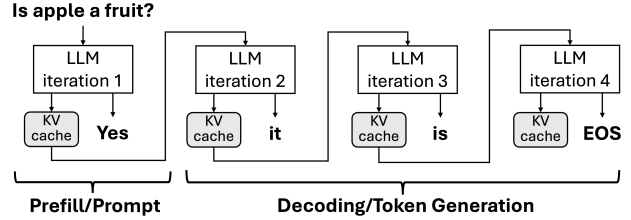


Figure 1: An LLM serving example.

and GPU lifetimes, even in low-carbon intensity settings (17 gCO₂/kWh). We summarize the contributions of this paper as follows:

- We leverage GPU heterogeneity to explore the reusing of older GPUs for LLM serving to reduce carbon emissions.
- We identify two use cases, Disg-Pref-Decode and Disg-Spec-Decode, to enable the disaggregation of LLM serving on heterogeneous GPUs.
- We address high bandwidth constraints by introducing a disaggregated speculative decoding approach with low network bandwidth requirements.
- We theoretically analyze the carbon emissions of disaggregated serving, highlighting its relationship with carbon intensity and GPU lifetime.
- We design and implement GreenLLM, an SLO-aware LLM serving system that minimizes carbon emissions by reusing old GPUs. We evaluate GreenLLM using three LLM serving datasets, validating our theoretical analysis.

2 Background and Related Work

This section first introduces the LLM inference process. We then review recent work on disaggregated serving and speculative decoding as potential use cases. Then, we discuss carbon accounting methods for operational and embodied emissions, followed by a literature survey of recent carbon reduction efforts in various applications and systems.

2.1 LLM Serving Process

LLMs are deep learning generative models based on transformer architecture [43]. Transformers use attention and multi-layer perceptron layers to process inputs and generate outputs. Figure 1 shows an example of LLM serving. Upon receiving a prompt request, all input tokens are processed in parallel within a single iteration to generate the first token. We call this the *prefill* or prompt processing phase. The context generated by the attention layers during prefill is stored in the key-value (KV) cache [40, 49], as it is required for all subsequent token generation iterations. After the first token is generated, the following tokens are generated using only the last generated token and the KV-cache as inputs for the model’s forward pass. We call this the *decoding* or token gen-

eration phase, which demands more memory bandwidth and capacity compared to the computationally intensive prefill phase. Furthermore, each phase has its own specific latency SLOs and preferences for different types of parallelism [50].

Performance metrics. LLM serving performance is mainly evaluated using two latency metrics: TTFT (time-to-first-token) and TPOT (time-per-output-token) [3, 32, 34, 50]. TTFT quantifies the time it takes to generate the first output token after a request enters the system, reflecting the model’s responsiveness during the prefill phase. TPOT measures the time interval between generating consecutive output tokens, which represents performance during the decoding phase.

2.2 LLM Serving Performance Optimization

Various techniques and open-source platforms, such as vLLM [23], have been developed to accelerate LLM serving. Some techniques, like request batching [4, 49] and parallelism [8], have become standard. Here, we review the following two techniques that we identify as use cases.

Disaggregated prefill and decoding. This method improves LLM serving performance by assigning the prefill and decoding phases to separate GPUs, rather than co-locating them [16, 22, 34, 50]. This approach offers two benefits: (1) eliminating interference between the two phases, and (2) enabling tailored resource allocation and parallelism strategies for each phase. However, disaggregation requires high network bandwidth to transfer the KV cache between GPUs. While modern GPU clusters with high-bandwidth networking like NVLink or Infiniband can mitigate this overhead, such environments may not always be practical. Furthermore, while prior works use multiple GPUs, *they solely rely on high-end GPUs and neglect the potential of using older GPUs to reduce embodied carbon emissions.*

Speculative decoding. This method consists of three primary components: the *draft model*, the *target model*, and the *verifier* [24, 30]. The draft model is a smaller, more efficient model that generates a sequence of speculative tokens, while the target model, which is larger and more capable, takes these speculative tokens and performs decoding to compute the probability distribution. Finally, the verifier decides whether to accept the tokens proposed by the draft model by comparing the probabilities from both models, typically using a rejection sampling scheme. This draft-then-verify paradigm enables simultaneous decoding of multiple tokens per step, significantly accelerating inference compared to the traditional autoregressive decoding.

The verifier employs a rejection sampling mechanism to determine whether to accept speculative tokens. Specifically, given a sequence of tokens $\{x_1, \dots, x_n\}$ and a set of K draft

tokens $\{\tilde{x}_{n+1}, \dots, \tilde{x}_{n+K}\}$ generated from the draft model’s conditional distribution $p(\cdot|x_1, \dots, x_n)$, the token $\tilde{x}_{n+1}$ is accepted with probability:

$$\min\left(1, \frac{q(\tilde{x}_{n+1}|x_1, \dots, x_n)}{p(\tilde{x}_{n+1}|x_1, \dots, x_n)}\right),$$

where $q(\tilde{x}_{n+1}|x_1, \dots, x_n)$ and $p(\tilde{x}_{n+1}|x_1, \dots, x_n)$ are the probabilities of the token $\tilde{x}_{n+1}$ from target and draft models, respectively, conditioned on the prior tokens in the sequence. Thus, there is a temporal dependency between these components: the target model relies on the speculative tokens predicted by the draft model, and the verifier depends on the probability outputs from both models. Prior work co-locates both models on the same GPU, *neglecting the potential of assigning the draft model to an older, less resource-intensive GPU to further optimize for carbon emissions.*

Summary. While various approaches have been introduced to optimize LLM service, they focus on improving performance and using high-end GPUs. This paper aims to explore how disaggregation techniques on heterogeneous GPUs can be harnessed to reduce total carbon emissions.

2.3 Carbon Emissions Accounting

According to the Greenhouse Gas Protocol [36], carbon emissions are classified into three categories: Scope 1 (direct emissions), Scope 2 (indirect emissions from energy consumption), and Scope 3 (indirect emissions from the production and transportation of goods procured). Following the terminology used in prior work [18, 19], we refer to Scope 1 as direct carbon emissions, Scope 2 as operational carbon emissions, and Scope 3 as embodied carbon emissions. In this paper, the total carbon emissions that we model and evaluate is the sum of embodied and operational carbon emissions.

Embodied carbon. The embodied carbon of a hardware device C_e is modeled based on the processor chip areas and memory capacities [14, 18]. To illustrate, we compare the specifications and embodied carbon emissions of three GPU types—T4, V100, and A100—in Table 1. For the rest of this paper, we will conduct experiments using these three GPU types. The A100, the latest-generation GPU, has the greatest compute capability, semiconductor technology, the largest chip area, and memory capacity, leading to the highest embodied carbon. T4 and V100 were introduced earlier and have the same memory capacity. Because V100 has a larger chip area, it has higher embodied carbon than T4. As such, the choice of GPU type can significantly influence the embodied carbon emission of a computing task.

The embodied carbon emission of a request is amortized over time by the ratio of the total execution time to the hardware lifetime (L_T) [18]. The lifetime of a GPU is 5-7

Table 1: **Specifications of GPUs in this paper.**

GPU Model	T4	V100	A100
VRAM (GB)	16	16	40
Memory Bandwidth (GB/s)	320	900	1555
Chip Area (mm ²)	545	815	826
Max Power (W)	70	300	400
Technology Node (nm)	12	12	7
FP16 TFLOPs	65	28.26	312
Year	2018	2017	2020
Embodied Carbon (kgCO₂)	10.3	20	26.34

years [33]. Therefore, a request executed on a GPU for t_{req} time generates the embodied carbon emission of:

$$C_{\text{req,e}} = \frac{t_{\text{req}}}{\text{LT}} \cdot C_e \quad (1)$$

Operational carbon. The operational carbon emission of a request $C_{\text{req,o}}$ is determined by the product of its energy consumption E_{request} and carbon intensity of the grid (CI). *Carbon intensity* is measured as grams of $\text{CO}_{2\text{eq}}$ emitted per kWh of electricity generated or consumed [25, 29]. Therefore, the operational carbon emission of a request is:

$$C_{\text{req,o}} = E_{\text{req}} \cdot \text{CI} \quad (2)$$

Total carbon. The total carbon emission consists of embodied and operational carbon emissions. For a request that executes for t_{request} time, the total carbon emission is:

$$C_{\text{req}} = C_{\text{req,e}} + C_{\text{req,o}} = \frac{t_{\text{req}}}{\text{LT}} C_e + E_{\text{req}} \cdot \text{CI} \quad (3)$$

2.4 Carbon Emission Reduction

There are two main lines of research aimed at reducing carbon emissions: one focuses on reducing operational carbon and the other on reducing embodied carbon.

Reducing operational carbon focuses on load shifting based on carbon intensity across regions and times [37]. Carbon Explorer provides a holistic framework for reducing carbon emissions for datacenters [2]. Ecovisor introduces a carbon-efficient virtual energy management system [41]. Carbon-Scaler uses workload elasticity to reduce operational carbon [20]. GAIA optimizes the cost of reducing cloud carbon emissions [21]. Caribou reduces carbon emissions from serverless applications through geospatial shifting [17].

Reducing embodied carbon focuses on extending hardware lifetimes. Junkyard Computing repurposes discarded smartphones to reduce carbon footprints for conventional cloud applications with low compute intensity, such as microservices [42]. GreenSKU reconfigures old, functional non-GPU hardware devices in datacenters, such as DRAMs and SSDs, for new servers [45]. However, these approaches target

reusing non-GPU hardware for low-compute-intensive workloads and are not directly applicable to LLM serving, which demands significantly higher compute capabilities.

Recently, some work has begun exploring sustainable LLM serving through carbon characterization and analysis [10, 14, 27, 31]. Despite these efforts, to the best of our knowledge, there are no system-level solutions for carbon reduction in LLM serving, nor do they leverage disaggregation techniques to reuse older GPUs [26].

3 Motivation

Serving LLMs on new GPUs incurs significant embodied and total carbon emissions. To reduce embodied carbon, we explore the potential of offloading a portion of the computation to older GPUs. However, this approach faces two technical challenges. First, offloading parts of the LLM model, such as the decoding phase, to older GPUs may lead to performance degradation. Second, the communication overhead between new and old GPUs, particularly when transferring KV cache, can be substantial. In this section, we characterize these challenges to identify potential solutions.

For the following experiments, we use randomized texts that match the input and output token lengths, as our performance and carbon emissions evaluations are independent of the text but only concerned with token lengths. We test on three GPU types: A100, V100, and T4.

3.1 Model Size and Phase Matter

We characterize latency and energy consumption for both the prefill and decoding phases of the Llama model in various sizes (300M, 1B, and 7B parameters). We take input and output token lengths of 160 and 140 on different GPUs. The input and output token lengths are highly representative as they follow the median lengths in the ShareGPT dataset [38].

Latency. Figure 2 shows the latency of the prefill (TTFT) and decoding (TPOT) of different model sizes at different request rates across different GPU types as a function of request rate. The latency SLOs for TTFT and TPOT are 200ms and 80ms, respectively. The results highlight three key findings. First, as the model size increases, both TTPT and TPOT increase across all GPU types due to greater compute demands, leading to longer processing times. Second, larger model sizes reduce the request rate that can meet latency SLOs, with the largest models (7B) potentially failing to meet SLOs at any request rate. Third, at many request rates, older GPUs (V100 and T4) can meet latency SLOs comparable to newer GPUs (A100), particularly for smaller models, revealing opportunities for running smaller models using older GPUs effectively.

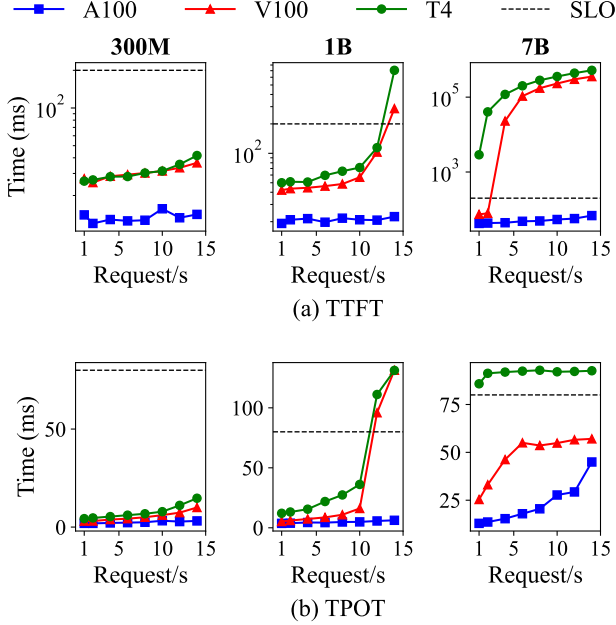


Figure 2: **TTFT (prefill) and TPOT (decoding) of different model sizes at different request rates on different GPUs.** The latency SLOs for prefill (TTF) and decoding (TPOT) phases are 200ms and 80ms, respectively.

Takeaway 1: The prefill and decoding phases feature different characteristics, as prior works have shown [35, 50]. Particularly for old GPUs, the prefill phase is more demanding as it is compute-bound, while the decoding phase is less demanding as it is memory-bound. Thus, disaggregating LLM serving enables using older GPUs for specific phases.

Energy. Figure 3 shows the energy consumption per token of different model sizes on different GPUs as a function of the request rate. The results highlight three key findings. First, energy increases with model size across all GPU types at the same request rate due to higher processing times and greater compute demands. Second, as request size grows, energy per token generally decreases. This is because larger requests improve GPU utilization, allowing the GPU to operate closer to its thermal design power (TDP). At this point, token throughput increases faster than power usage, stabilizing energy consumption per token when the GPU reaches optimal throughput within its TDP. Third, older GPUs (V100, T4) tend to be more energy-efficient than newer GPUs (A100) for smaller models (300M, 1B). However, this advantage diminishes for larger models like 7B, where older GPUs are only more energy-efficient at low request rates.

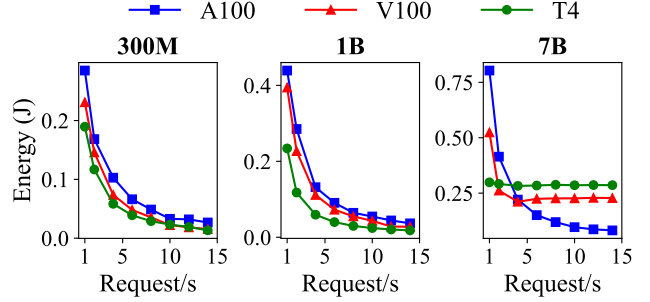


Figure 3: **Energy per token of different model sizes at different request rates on different GPUs.**

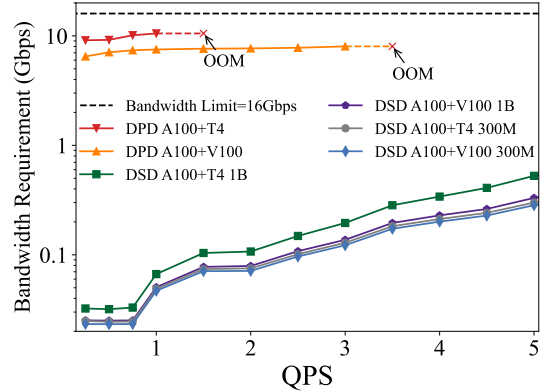


Figure 4: **Bandwidth requirement comparisons between Disg-Pref-Decode (DPD) and Disg-Spec-Decode (DSD) when connecting A100 and T4/V100 at different request rates (QPS).** DPD experiments the 7B model only, with the prefill phase running on A100 and the decoding phase on T4/V100. For DSD, the 7B target model runs on A100, while the 1B or 300M draft model runs on T4/V100.

Takeaway 2: Small-size models can meet comparable latency SLOs as larger models at certain request rates, even on older GPUs, while providing significantly better energy efficiency. To meet latency SLOs, offloading computations to older GPUs is more effective in executing smaller models. For less complex LLM tasks, replacing large models on new GPUs with smaller ones on older GPUs can be more energy- and carbon-efficient.

3.2 Bandwidth Requirement Matters

As discussed in §2.2, a key limitation of disaggregating the prefill and decoding phases is the high interconnect bandwidth requirement due to the need for transferring large KV caches. Although prior work like DistServe [50] and Splitwise [34] have discussed this issue, their solutions depend on high-end homogeneous or heterogeneous GPUs equipped with high-

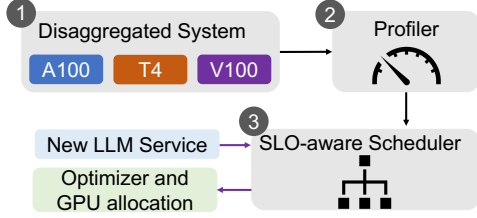


Figure 5: **Workflow of GreenLLM.**

speed interconnects such as NVLink or InfiniBand. However, this limitation persists in scenarios where high-bandwidth connectivity is unavailable and becomes even more pronounced when data transfers occur between heterogeneous GPUs—where one is high-end and the other is older and lower-tier. In such cases, performance gains diminish and carbon reduction benefits are also reduced.

Figure 4 illustrates the bandwidth requirements when connecting heterogeneous GPUs, such as A100 and T4/V100, with a 16Gbps bandwidth cap. The results show that when serving the LLaMA 7B model and transferring the KV cache between an A100 and T4/A100, the system quickly encounters out-of-memory (OOM) issues as QPS increases.

To address this issue, we identify a second use case based on speculative decoding. In the experiments, the target model is 7B, while the draft models are smaller (1B and 300M). As shown in Figure 4, Disg-Spec-Decode requires $65\text{--}434\times$ lower bandwidth than Disg-Pref-Decode over a wide range of request rates since it only transfers tokens and corresponding probability distributions rather than the large KV caches.

Takeaway 3: Disaggregating prefill and decoding phases on different GPUs is feasible only when high network connectivity is available. Otherwise, Disg-Spec-Decode is an alternative for reducing carbon emissions. Combining the results from §3.1, running the draft model of speculative decoding on older GPU is viable and offers potential for high carbon efficiency.

4 GreenLLM

We present GreenLLM, an SLO-aware LLM serving framework that minimizes carbon emissions using disaggregation on heterogeneous GPUs. Figure 5 shows the workflow of GreenLLM, which consists of three components: ① a disaggregated system on heterogeneous GPUs, ② a profiler, and ③ an SLO-aware scheduler. ① implements how we disaggregate two optimizers (Disg-Pref-Decode and Disg-Spec-Decode) on heterogeneous GPUs with minimal communication overhead. ② performs profiling to measure GPU performance and carbon emissions across request sizes and rates. ③ selects the optimal optimizer and GPU configuration to minimize carbon emissions using profiled data. Next, we will elaborate on the

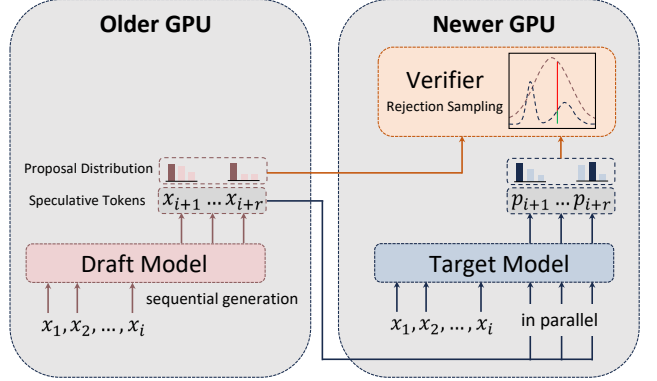


Figure 6: **Disaggregating draft and target models on heterogeneous GPUs for speculative decoding in GreenLLM.**

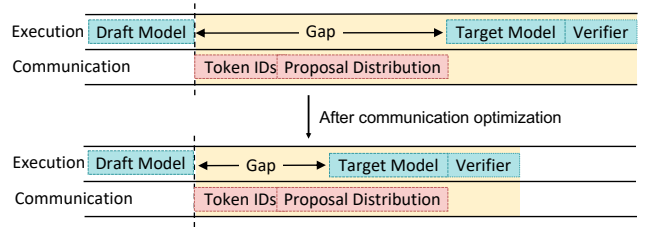


Figure 7: **Communication overlapping optimization for Disg-Spec-Decode.** Green blocks indicate the execution of each component. Red blocks indicate data transfer. The yellow background indicates the time comparisons.

design and implementation of each component.

4.1 Disaggregation on Heterogeneous GPUs

As GreenLLM integrates two LLM serving optimizers (Disg-Pref-Decode and Disg-Spec-Decode), each optimizer disaggregates different execution components onto different GPUs. Consequently, their disaggregation designs differ. We will elaborate on each design as follows.

Disg-Pref-Decode. Because the decoding phase is less compute-intensive than the prefill, Disg-Pref-Decode allocates the prefill phase to newer GPU and the decoding phase to older GPU while meeting latency SLOs. Disg-Pref-Decode differs from prior disaggregation work like DistServe [50] and Splitwise [34] in two ways. First, prior work disaggregated both phases on high-end GPUs (e.g., H100, A100), neglecting the potential of reusing old, low-performing GPUs. Second, while prior work prioritized performance and cost optimization, GreenLLM focuses on carbon reduction. Although we identify Disg-Pref-Decode as a use case for reusing old GPUs, as indicated in §3.2, the high network bandwidth requirement of this disaggregation strategy limits its applicability in scenarios with limited network connectivity, leading to our second optimizer introduced below.

Disg-Spec-Decode. As discussed in §3.1, smaller models can achieve SLOs comparable to larger models, even on older

GPUs, while providing better energy efficiency. Based on this observation, Disg-Spec-Decode assigns the draft model to an older GPU and the target model (along with the verifier) to a newer GPU, as illustrated in Figure 6, thus reducing energy consumption and carbon emissions. However, separating the draft and target models across GPUs inevitably introduces communication overhead.

To maximize carbon savings while minimizing this overhead, we analyze the temporal dependencies between the two models to explore the opportunity for communication optimization. We observe that the speculative token IDs and the draft model’s probabilities differ significantly in size, with the probabilities being V times larger, where V is the vocabulary size. Importantly, the verifier does not require the draft probabilities until after the target model has processed the speculative tokens.

Based on this insight, we illustrate the communication optimization process in Figure 7. We first transfer only the speculative token IDs, which are relatively small in size. While the target model performs decoding using these token IDs, we overlap communication and computation by asynchronously transferring the larger draft probabilities in parallel. This approach minimizes communication overhead by leveraging the delayed dependency of probabilities, effectively hiding data transfer delays between the draft and target models.

4.2 Profiler

GreenLLM includes a profiler to collect the latency, energy consumption, and carbon emissions for different GPUs across a range of LLM serving workloads (e.g., varying request sizes and rates). For Disg-Pref-Decode, it collects data for the prefill and decoding phases separately running on different types of GPUs. For Disg-Spec-Decode, it collects data for different model sizes running on different types of GPUs. This profiling data will be used as the database for the SLO-aware scheduler described below.

4.3 SLO-Aware Scheduler

GreenLLM incorporates an SLO-aware scheduler to identify the optimal *configuration*—the combination of an LLM serving optimizer and its GPU allocation—to execute a given LLM service workload with specified request size and query rate (queries per second or QPS). Before initiating the search, missing latency and carbon emission values for unseen request sizes and rates must be estimated.

To address the search across both workload and configuration dimensions, GreenLLM organizes the search space into two two-dimensional matrices: one matrix, $\mathbf{C}$, representing carbon emissions, and another, $\mathbf{SLO}_{att}$, representing SLO attainment. In both matrices, the row represents combinations of applications and QPS levels, and the column represents

Algorithm 1: The SLO-aware Scheduler

Input: Profiling database $\mathbf{D}$, serving workload characteristics $\mathbf{W} = \{(req_size, qps)\}$, SLO_{target} , priority
Output: Optimal configuration **Optimal** for each workload

```

1  $\mathbf{C}, \mathbf{SLO}_{att} \leftarrow \text{CollaborativeFiltering}(\mathbf{D})$ 
  foreach  $j \in \mathbf{W}$  do
2    $\text{Feasible}_j \leftarrow \{i \mid \mathbf{SLO}_{att}[i, j] \geq SLO_{target}\}$ 
3   if  $\text{Feasible}_j \neq \emptyset$  then
4      $\text{Optimal}[j] \leftarrow \arg \min_{i \in \text{Feasible}_j} \mathbf{C}[i, j]$ 
5   else
6      $\text{Optimal}[j] \leftarrow \text{FallbackStrategy}(\mathbf{SLO}_{att}, \mathbf{C}, j, \text{priority})$ 
7 return Optimal
8 Procedure  $\text{FallbackStrategy}()$ 
9   if  $\text{priority} = 'SLO'$  then
10     $\text{return } \arg \max_i \mathbf{SLO\_Att}[i, j]$ 
11  else
12     $\text{return } \text{Default configuration}$ 
```

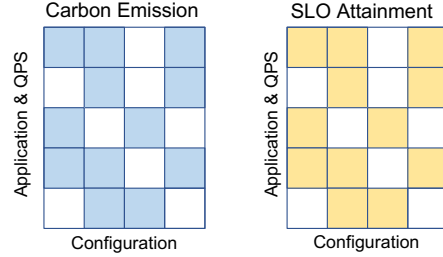


Figure 8: **Matrix formulation for carbon emissions and SLO attainment.** Shaded regions represent known data, while blank entries represent missing data. Each entry represents the carbon emission or SLO attainment for a specific application, QPS level, and configuration. GreenLLM uses collaborative filtering to fill in the missing entries.

configurations. The entries in these matrices are carbon emissions and SLO attainment, respectively. Figure 8 illustrates the two matrices with missing values. To fill in missing values in these matrices, GreenLLM employs collaborative filtering, a technique previously used in resource management [12, 13]. This process is performed only once after profiling.

Once the matrices are complete, GreenLLM searches for the configuration that minimizes carbon emissions while meeting the SLO. If no such configuration exists, a fallback strategy is applied based on the specified priority, such as maximizing SLO attainment or selecting a default configuration. The search algorithm is summarized in Algorithm 1.

5 Carbon Efficiency Analysis

This section delves into the carbon efficiency analysis of disaggregation, comparing scenarios with both new and old GPUs to that using only new GPU. We aim to answer three questions: 1) When does disaggregation yield greater carbon savings? 2) How does carbon intensity influence carbon savings? 3) How does GPU lifetime influence carbon savings?

Consider two cases that run the same LLM service.

Case 1: (Standalone) The LLM service runs on a new GPU A, incurring execution time t_A , energy consumption N_A , operational carbon emission O_A , and embodied carbon emission E_A .

Case 2: (Disaggregation) The same LLM service runs on a new GPU A and an old GPU B, incurring execution time t'_A, t_B , energy consumption N'_A, N_B , operational carbon emission O'_A, O_B , and embodied carbon emission E'_A, E_B .

Our analysis will be based on the following assumptions:

- A.1:** Both cases use electricity from the same power grid with carbon intensity α .
- A.2:** The carbon emissions of communication overhead between two GPUs in Case 2 are negligible.
- A.3:** The time difference between running A in Case 1 and Case 2 is substantially shorter than the time to run B in Case 2. This assumption is practical, as offloading computations to older GPUs typically incurs longer running times as well as higher total embodied carbon than running the same computations on newer GPUs.

When does disaggregation yield greater carbon savings?

To reduce total carbon emissions in Case 2, we need

$$\begin{aligned} O_A + E_A &> O'_A + E'_A + O_B + E_B \\ \Rightarrow O_A - (O'_A + O_B) &> (E'_A + E_B) - E_A \end{aligned}$$

Due to A.3, $(E'_A + E_B) - E_A > 0$, as disaggregation, involving two GPUs, typically incurs higher embodied carbon than standalone. Therefore, we have

$$\begin{aligned} O_A - (O'_A + O_B) &> 0 \\ \Rightarrow O_A &> O'_A + O_B \\ \Rightarrow E_A \cdot \alpha &> (E'_A + E_B) \cdot \alpha \\ \Rightarrow E_A &> E'_A + E_B \end{aligned} \quad (4)$$

Carbon Implication 1: Equation (4) indicates that disaggregated systems must consume less energy than standalone systems to achieve carbon savings.

How does carbon intensity influence carbon savings? To

quantify carbon savings, we can calculate

$$\begin{aligned} \frac{O'_A + E'_A + O_B + E_B}{O_A + E_A} &= \frac{(N'_A + N_B) \cdot \alpha + E'_A + E_B}{N_A \cdot \alpha + E_A} \\ &= \frac{\frac{N'_A + N_B}{N_A} (N_A \cdot \alpha + E_A - E_A) + E'_A + E_B}{N_A \cdot \alpha + E_A} \\ &= \frac{N'_A + N_B}{N_A} + \frac{E'_A + E_B - \frac{N'_A + N_B}{N_A} E_A}{N_A \cdot \alpha + E_A} \end{aligned} \quad (5)$$

Carbon Implication 2: Equation (5) indicates that carbon savings increase with increasing carbon intensity α and decrease with decreasing α .

How does GPU lifetime influence carbon savings? Let $\mathcal{A}$ and $\mathcal{B}$ be the total embodied carbon emissions of GPU A and B, respectively. T_A and T_B are the lifetimes of the GPUs A and B, respectively. We keep deriving Equation (5) and simplify it by removing constants, and have

$$\begin{aligned} \frac{O'_A + E'_A + O_B + E_B}{O_A + E_A} &\approx \frac{E'_A + E_B - \frac{N'_A + N_B}{N_A} E_A}{N_A \cdot \alpha + E_A} \\ &= \frac{\frac{t'_A}{T_A} \mathcal{A} + \frac{t_B}{T_B} \mathcal{B} - \frac{N'_A + N_B}{N_A} \frac{t_A}{T_A} \mathcal{A}}{N_A \cdot \alpha + \frac{t'_A}{T_A} \mathcal{A}} \\ &= \frac{\left(\frac{t'_A}{T_A} - \frac{N'_A + N_B}{N_A} \frac{t_A}{T_A} \right) \mathcal{A} + \frac{t_B}{T_B} \mathcal{B}}{N_A \cdot \alpha + \frac{t'_A}{T_A} \mathcal{A}} \\ &\approx \text{Const.} + \frac{\frac{t_B}{T_B} \mathcal{B}}{N_A \cdot \alpha + \frac{t'_A}{T_A} \mathcal{A}} \end{aligned} \quad (6)$$

When fixing T_B , decreasing T_A will increase carbon savings. When fixing T_A , increasing T_B will increase carbon savings.

Carbon Implication 3: Equation (6) indicates disaggregation yields maximum carbon savings when combined with newer GPUs that have been in service for a short time and older GPUs that have been operating for a long time.

In summary, disaggregation systems achieve the highest carbon savings under high carbon intensity, high energy efficiency, and a combination of shorter service time for newer GPUs and longer service time for older GPUs.

6 Implementation

We implemented GreenLLM on top of the open-source LLM serving platform vLLM [23], integrating the profiler and Disg-Spec-Decode method as plug-ins. For the Disg-Pref-Decode method, we leveraged community implementations of vLLM based on the design principles of Splitwise [34].

Table 2: **Datasets for evaluation and SLO requirements.**

Dataset	Task	TTFT SLO	TPOT SLO	P25 Req. Size	P50 Req. Size	P75 Req. Size
ShareGPT [38]	Chatbot	200ms	80ms	(24,24)	(160,140)	(510,357)
HumanEval [9]	Code Completion	125ms	200ms	(108,31)	(136,55)	(182,88)
LongBench [6]	Summarization	15s	150ms	(1134,201)	(1495,275)	(1817,352)

The profiler uses `pynvml` APIs to collect GPU power consumption data with minimal overhead. Communication and profiling data aggregation across multiple servers are managed via RESTful APIs, ensuring robustness and efficiency in distributed environments. The profiler is implemented in 1.7K lines of Python and Linux Shell code.

The Disg-Spec-Decode method was implemented on top of vLLM with an additional 1K lines of Python code. Following vLLM’s architecture, we utilized Ray actors to create GPU workers for the draft model inference, assigning older-generation GPUs in a heterogeneous cluster for running the draft model workers. Unlike the original vLLM design, we introduced two distinct parallelism groups on heterogeneous nodes to support parallel inference. One group was responsible for the draft model, while the other handled the primary serving instance, including the serving API, target model, and verifier. To facilitate communication between these groups, we employed NCCL along with PyTorch’s asynchronous communication primitives, `isend` and `irecv`, to avoid blocking GPU computations during data transmission.

7 Evaluation

We present a comprehensive evaluation as follows. First, we show the main results of carbon emission savings achieved by GreenLLM under various workloads (§7.2). Second, we present the latency results of GreenLLM (§7.3). We then show the impact of network bandwidth on disaggregation and carbon savings (§7.4), followed by sensitivity analyses of varying carbon intensity (§7.5) and GPU lifetime (§7.6).

7.1 Methodology

In this section, we discuss the system setup, models and configurations, workloads, measurement approach, and metrics.

System setup. We evaluate GreenLLM using Google Cloud Platform (GCP) servers: machine type `a2-highgpu-1g` with one A100 GPU, `n1-standard-8` with one T4/V100 GPU. When serving in a disaggregated mode, servers are connected within the GCP network with a default bandwidth of 16Gbps. **Models and configurations.** We evaluate GreenLLM with Llama models of varying sizes: 7B, 1B, and 300M parameters [5]. GreenLLM incorporates the following configurations:

- **Standalone:** serve a 7B Llama model on a single A100.

- **SpecDecode** (Single-GPU Speculative Decoding): serve target (7B) and draft (1B or 300M) models from speculative decoding on a single A100.
- **DPD** (Disg-Pref-Decode): serve prefill and decoding phases of a 7B Llama model on different GPUs: the prefill stage on newer GPU (A100), and the decoding stage on older GPUs (T4 or V100).
- **DSD** (Disg-Spec-Decode): use 1B and 300M Llama models as choices of draft model running on old GPUs (T4 or V100), and 7B Llama model as the target model on newer GPU (A100).

GreenLLM automatically selects these configurations to minimize carbon emissions and meet latency SLOs. We take the Standalone A100 setup as *baseline*, representing a system that does not reuse old GPUs but only uses newer GPUs.

Workloads. We evaluated the same LLM serving datasets as in previous work DistServe [50]: ShareGPT [38], HumanEval [9], and LongBench [6]. They represent three types of LLM applications: ChatGPT-like chatbot, code completion, and summarization, respectively. Table 2 summarizes the SLOs, and input and output distributions for each workload. When evaluating a fixed input/output token length for a workload, we truncate the prompts to the specific input length and limit the output length. This way, the carbon emission and performance results of different prompts are comparable.

Measurement and metrics. We measure execution time by inserting a timer in GreenLLM and record token generation timestamps. During model execution, we measure the GPU power consumption every 200 ms using the `pynvml` library. Carbon emissions are calculated using the carbon intensity of 261 gCO₂/kWh from the CISO grid by default.

We evaluate GreenLLM using the following metrics:

- **Carbon Per Token:** gCO₂ of carbon emission per token. We model carbon emissions based on the carbon accounting method in §2.3.
- **Time To First Token (TTFT):** the latency between prompt reception and the generation of the first token.
- **Time Per Output Token (TPOT):** the latency between two consecutive tokens.

We evaluate these metrics by varying the query per second (QPS) and input/out sizes conditions to holistically assess the carbon efficiency of GreenLLM.

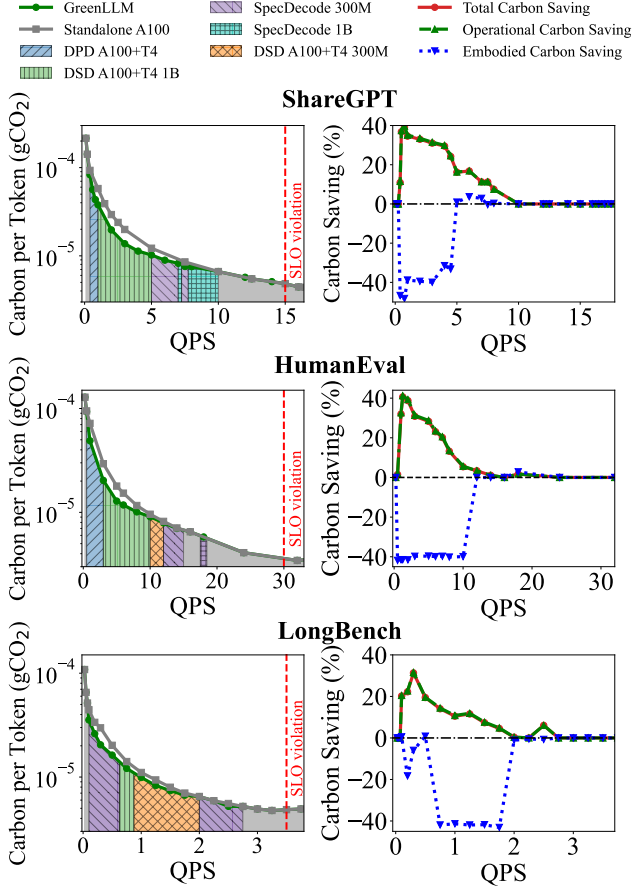


Figure 9: **Carbon per token and carbon savings comparison between GreenLLM and the baseline Standalone A100.** The left shows GreenLLM’s optimal configurations at various QPS. The right shows the corresponding total carbon savings breakdown into operational and embodied emissions.

7.2 Carbon Emission Savings

In this section, we evaluate GreenLLM’s carbon emissions and savings. For each workload, we assess carbon emissions and savings under varying QPS and input/output sizes. We also analyze the breakdown of embodied and operational carbon emissions improvement.

Variable QPS. Figure 9 shows the carbon emission result, where the left column shows the carbon per token under different QPS values while the right column shows the corresponding carbon emission savings per token compared to the standalone A100 baseline. The input and output sizes of requests follow the median value of each dataset. Overall, GreenLLM can effectively save up to 31.3–40.6% carbon emissions when SLO attainment is 90% (i.e., meeting SLO 90% of the time). Carbon emission savings occur when the QPS lies in the lower range of each workload. In particular, we notice that QPS ranges that lead to the highest carbon emission savings vary among the three workloads. LongBench

features the highest input/output token numbers, therefore GreenLLM benefits from older GPUs only under lower QPS range [0.75,2]. HumanEval is on the contrary as it has lower input/output tokens, and thus old GPUs can provide benefits across a wider QPS range [0.5,11].

Variable input/output sizes. We next investigate the carbon savings upon different input/output sizes. ShareGPT [38] is a highly representative dataset as it is based on prompts from real-world users. Therefore, we further evaluate token lengths at the three percentiles: P25, P50, and P75. Figure 10 the per-token carbon emissions and carbon savings from GreenLLM under these lengths. Overall, under the same QPS, as the input/output sizes increase, the per-token carbon emission goes down as the carbon emissions are amortized among the tokens. At the same time, as the size increases, the QPS range that yields carbon savings becomes lower. This is because larger input sizes have higher computation loads and cause older GPUs to be less efficient.

Carbon savings breakdown. To further analyze the specific components of GreenLLM’s carbon savings, we show the breakdown of the savings from operational and embodied carbon in the second column of Figure 9. At lower QPS, the primary source of carbon savings stems from reductions in operational carbon within disaggregated modes. This is because using older GPUs in disaggregated modes increases embodied carbon. As QPS increases, GreenLLM prioritizes the SpecDecode method on a standalone A100, resulting in significant reductions in embodied carbon. This shift is driven by the fact that at higher QPS, using smaller draft models to speculatively generate output tokens leads to substantial improvements in TPOT, ultimately resulting in lower end-to-end latency. As indicated by Equation 1, lower latency translates to reduced embodied carbon. In conclusion, this experiment demonstrates that GreenLLM can effectively reduce both operational and embodied carbon by selecting the optimal configuration across a wide range of QPS.

7.3 Latency

In this section, we evaluate the latency of GreenLLM and its SLO attainment under variable QPS.

Variable QPS. Figure 11 shows the TTFT and TPOP latencies of GreenLLM and the standalone A100 baseline under variable QPS. GreenLLM automatically reconfigures the GPUs to meet SLO (the red line). Therefore, GreenLLM meets both the TTFT and TPOP SLOs unless the QPS exceeds the capability of the standalone A100. In lower QPS ranges, GreenLLM can experience a higher latency than standalone A100 because of old GPU reuse but still remains below SLO. On the other hand, Figure 9 demonstrates substantial carbon emission reductions during these SLO ranges, indicating that the GreenLLM is effective at reconfiguring the GPUs to minimize carbon emissions while meeting SLOs.

SLO attainment. Figure 12 compares the SLO attainment of

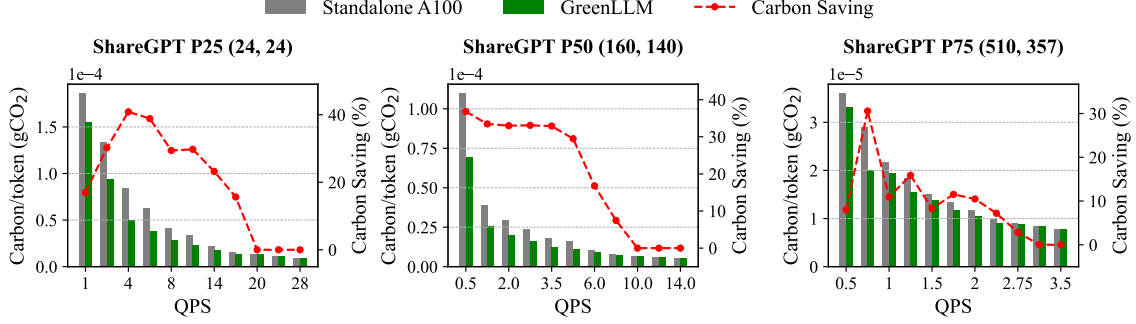


Figure 10: Carbon emission per token and Savings at different request sizes on the ShareGPT dataset.

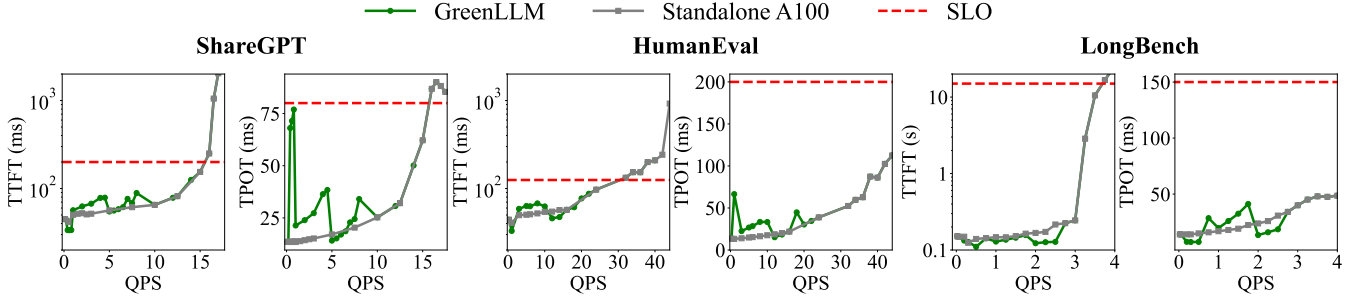


Figure 11: Latency (TTFT and TPOT) at various QPS across three datasets.

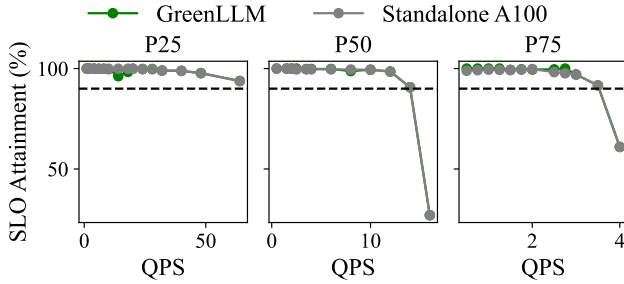


Figure 12: SLO attainment at three different request sizes on the ShareGPT dataset. The black dashed horizontal line indicates the SLO attainment of 90%.

GreenLLM and a standalone A100 configuration for three different request sizes (P25, P50, and P75) from the ShareGPT dataset. The black dashed horizontal line indicates the 90% SLO threshold. The results show that GreenLLM can effectively handle different request sizes while maintaining SLO performance similar to the standalone A100, but with the added benefit of greater carbon savings (Figure 10).

7.4 Impact from Network Bandwidth

As discussed in §3.2, network bandwidth is crucial for efficient disaggregation and carbon reduction. This experiment

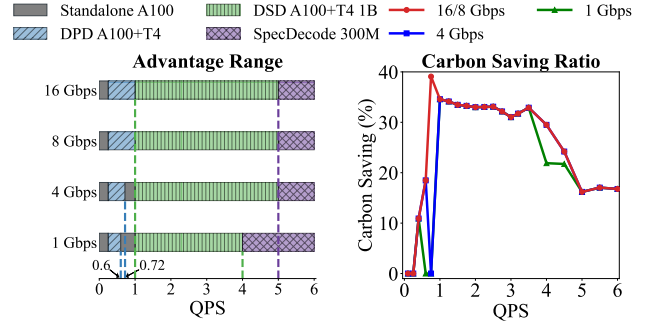


Figure 13: Optimal configurations and the corresponding carbon savings of GreenLLM across different QPS and network bandwidth.

investigates its impact on carbon savings by using ShareGPT dataset and controlling the request size to the median. The left side of Figure 13 summarizes the range of QPS under which each configuration achieves carbon savings across a range of network bandwidths, from 1 Gbps to 16 Gbps. The right side of Figure 13 shows the corresponding carbon savings. Across all bandwidths, Disg-Spec-Decode and SpecDecode consistently provide the highest carbon savings when QPS exceeds 1. In particular, SpecDecode performs best at a bandwidth of 1 Gbps. This is because these configurations do not require high bandwidth and, consequently, can operate effec-

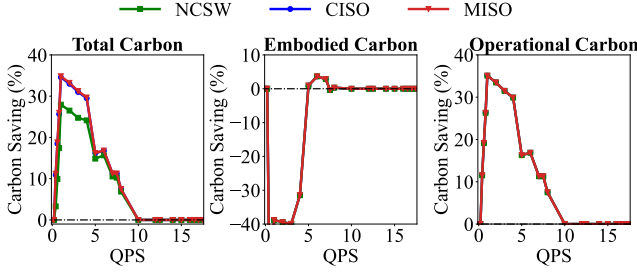


Figure 14: **Carbon saving of GreenLLM in different geographical regions with varying carbon intensity.** NCSW, CISO, and MISO have carbon intensities of 17, 261, and 501 gCO₂/kWh, respectively.

tively at high QPS rates. In contrast, Disg-Pref-Decode and Standalone, which rely on high bandwidth, are most effective at low QPS rates. Based on these findings, we conclude that when network bandwidth is limited, configurations involving speculative decoding, particularly disaggregated versions, can effectively use older GPUs.

7.5 Impact from Carbon Intensity

To validate the consistency of GreenLLM’s carbon savings across different geographical regions and to evaluate its performance under varying carbon intensity levels, we conduct an analysis using the ShareGPT P50 request size and three power grids: North Central Sweden (NCSW), California (CISO), and Midcontinent (MISO). These regions represent low, medium, and high carbon intensity levels, with carbon intensities of 17, 261, and 501 gCO₂/kWh, respectively. Figure 14 shows the total carbon savings and a breakdown of these savings for GreenLLM in each of these regions. Notably, GreenLLM selects the same optimal configuration across all three regions, regardless of the significant differences in carbon intensity, leading to consistent embodied and operational carbon savings. As demonstrated by the analysis of Equation (5) in §5, higher carbon intensity in CISO and MISO results in greater carbon savings compared to NCSW. However, even in regions with low carbon intensity, such as NCSW, GreenLLM can still achieve carbon savings of up to 27.9%. The close carbon savings observed between CISO and MISO suggest that once the carbon intensity reaches a sufficiently high level, further increases in carbon intensity have a marginal impact on the overall savings. In conclusion, GreenLLM effectively translates improved energy efficiency into enhanced carbon efficiency across diverse geographical regions.

7.6 Impact from GPU Lifetime

We vary the expected lifetime of GPUs to show the impact on carbon emissions. Like before, we evaluate ShareGPT and control the input/output token length to its median. The

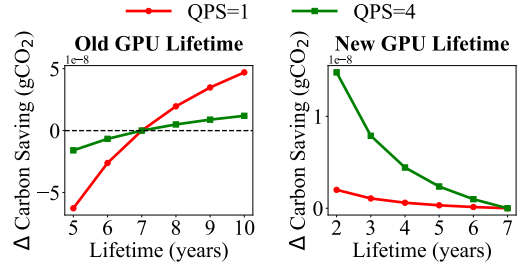


Figure 15: **Impact of GPU lifetime on carbon saving of DSD A100+T4 1B configuration.**

ShareGPT result in Figure 9 shows two interesting points: one at QPS=1 where the disaggregation configuration Disg-Spec-Decode becomes the optimal carbon-saving configuration and another at QPS=4 where it makes way for SpecDecode. Figure 15 zooms into these two data points. The left-side figure shows the expected carbon savings change compared the default 7-year lifetime old GPUs to the old GPU’s expected lifetime (5–10 years); the right-side shows the expected carbon savings change compared the default 7-year lifetime new GPUs to the new GPU’s expected lifetime (2–7 years). Overall, as the lifetime of old GPUs increases, the savings on carbon emissions increase. In comparison, as the lifetime of new GPUs decreases, the savings on carbon emissions reduce. This trend is expected as when the old GPU’s lifetime increases, its amortized embodied carbon per second decreases; when the new GPU’s lifetime decreases, its amortized embodied carbon per second increases. The newly installed GPU (A100) in this paper has a shorter lifetime compared to older GPUs (T4 and V100) that have been in service for years. This study validates the implications in §5 that short-lifetime new GPUs can work cooperatively with old GPUs to reduce their embodied carbon emissions.

8 Conclusion

The wide adoption of LLMs has led to significant environmental impacts due to their high computational intensity and resource demands. This demand has driven the development of new generations of high-performing GPUs, exacerbating the issue of electronic waste and premature disposal of devices. GreenLLM addresses the challenge of reducing the carbon emissions of LLM serving by reusing older, low-performing GPUs. By demonstrating the feasibility of reusing old GPUs and disaggregating workloads to balance carbon efficiency with performance goals, we hope to inspire future research into sustainable AI systems. Beyond LLM serving, the principles and methods introduced in GreenLLM could extend to other compute-intensive AI applications, encouraging broader adoption of sustainable computing practices.

References

- [1] The global e-waste monitor 2024. *International Telecommunication Union (ITU) and United Nations Institute for Training and Research (UNITAR)*, 2024.
- [2] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. Carbon explorer: A holistic framework for designing carbon aware datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2023.
- [3] Megha Agarwal, Asfandiyar Qureshi, Nikhil Sardana, Linden Li, Julian Quevedo, and Daya Khudia. LLM inference performance engineering: Best practices. <https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices>, 2023.
- [4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- [5] Meta AI. Llama: Open and efficient foundation language models. <https://ai.facebook.com/blog/large-language-model-llama>, 2024. Accessed: 2024-07-20.
- [6] Yushi Bai, Xin Lv, Jiajie Zhang, Hong Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding. *ArXiv*, abs/2308.14508, 2023.
- [7] Katherine Bourzac. Generative ai has a massive e-waste problem, 2024.
- [8] Felix Brakel, Uraz Odyurt, and Ana-Lucia Varbanescu. Model parallelism on distributed infrastructure: A literature review from theory to llm case-studies, 2024.
- [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, Suchir Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.
- [10] Andrew A Chien, Liuzixuan Lin, Hai Nguyen, Varsha Rao, Tristan Sharma, and Rajini Wijayawardana. Reducing the carbon impact of generative AI inference (today and in 2035). In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon)*, 2023.
- [11] Casey Crownhart. Ai will add to the e-waste problem. here’s what we can do about it. *MIT Technology Review*, 2024.
- [12] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. *ACM SIGPLAN Notices*, 48(4):77–88, 2013.
- [13] Yi Ding, Nikita Mishra, and Henry Hoffmann. Generative and multi-phase learning for computer systems optimization. In *Proceedings of the 46th International Symposium on Computer Architecture*, 2019.
- [14] Ahmad Faiz, Sotaro Kaneda, Ruhan Wang, Rita Chukwunyere Osi, Prateek Sharma, Fan Chen, and Lei Jiang. LLMCarbon: Modeling the end-to-end carbon footprint of large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [15] Sarah Griffiths. Why your internet habits are not as clean as you think. <https://www.bbc.com/future/article/20200305-why-your-internet-habits-are-not-as-clean-as-you-think>, 2020.
- [16] Tyler Griggs, Xiaoxuan Liu, Jiaxiang Yu, Doyoung Kim, Wei-Lin Chiang, Alvin Cheung, and Ion Stoica. Mélange: Cost efficient large language model serving by exploiting gpu heterogeneity, 2024.
- [17] Viktor Urban Gsteiger, Pin Hong Long, Yiran Sun, Parshan Javanrood, and Mohammad Shahradd. Caribou: Fine-grained geospatial shifting of serverless applications for sustainability. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024.
- [18] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S. Lee, David Brooks, and Carole-Jean Wu. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In *ISCA*, 2022.

- [19] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. Chasing carbon: The elusive environmental footprint of computing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021.
- [20] Walid A Hanafy, Qianlin Liang, Noman Bashir, David Irwin, and Prashant Shenoy. Carbonscaler: leveraging cloud workload elasticity for optimizing carbon-efficiency. *ACM SIGMETRICS Performance Evaluation Review*, 2024.
- [21] Walid A Hanafy, Qianlin Liang, Noman Bashir, Abel Souza, David Irwin, and Prashant Shenoy. Going green for less green: Optimizing the cost of reducing cloud carbon emissions. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [22] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate LLM inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *SOSP*, 2023.
- [24] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.
- [25] Amy Li, Sihang Liu, and Yi Ding. Uncertainty-aware decarbonization for datacenters. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*, 2024.
- [26] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. Toward sustainable GenAI using generation directives for carbon-friendly large language model inference, 2024.
- [27] Yueying Lisa Li, Omer Graif, and Udit Gupta. Towards carbon-efficient llm life cycle. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*, 2024.
- [28] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [29] Diptyaroop Maji, Prashant Shenoy, and Ramesh K Sitaraman. CarbonCast: Multi-day forecasting of grid carbon intensity. In *Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys)*, 2022.
- [30] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024.
- [31] Sophia Nguyen, Beihao Zhou, Yi Ding, and Sihang Liu. Towards sustainable large language model serving. In *Proceedings of the 3rd Workshop on Sustainable Computer Systems (HotCarbon)*, 2024.
- [32] Nvidia. NIM for LLM benchmarking guide – metrics. <https://docs.nvidia.com/nim/benchmarking/llm/latest/metrics.html>, 2024.
- [33] George Ostrouchov, Don Maxwell, Rizwan A. Ashraf, Christian Engelmann, Mallikarjun Shankar, and James H. Rogers. GPU lifetimes on titan supercomputer: Survival analysis and reliability. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [34] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024.
- [35] Tirthak Patel and Devesh Tiwari. Clite: Efficient and qos-aware co-location of multiple latency-critical jobs for warehouse scale computers. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020.
- [36] Greenhouse Gas Protocol. Greenhouse gas protocol. *Sector Toolsets for Iron and Steel-Guidance Document*, 2011.
- [37] Ana Radovanović, Ross Koningstein, Ian Schneider, Bokan Chen, Alexandre Duarte, Binz Roy, Diyu Xie, Maya Haridasan, Patrick Hung, Nick Care, Saurav Talukdar, Eric Mullen, Kendal Smith, MariEllen Cottman, and Walfredo Cirne. Carbon-aware computing for datacenters. *IEEE Transactions on Power Systems*, 2022.
- [38] ShareGPT. Sharegpt - share and save your conversations with ai. <https://sharegpt.com/>.

- [39] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [40] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [41] Abel Souza, Noman Bashir, Jorge Murillo, Walid Hanafy, Qianlin Liang, David Irwin, and Prashant Shenoy. Ecovisor: A virtual energy system for carbon-efficient applications. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2023.
- [42] Jennifer Switzer, Gabriel Marcano, Ryan Kastner, and Pat Pannuto. Junkyard computing: Repurposing discarded smartphones to minimize carbon. In *ASPLOS*, 2023.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 2017.
- [44] Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. Freshllms: Refreshing large language models with search engine augmentation. *Findings of the Association for Computational Linguistics ACL 2024*, 2024.
- [45] Jaylen Wang, Daniel S. Berger, Fiodar Kazhamiaka, Celine Irvine, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warriar, Chetan Bansal, Jonathan Stern, Ricardo Bianchini, and Akshitha Sriman. Designing cloud servers for lower carbon. In *ISCA*, 2024.
- [46] Peng Wang, Ling-Yu Zhang, Asaf Tzachor, and Wei-Qiang Chen. E-waste challenges of generative artificial intelligence. *Nature Computational Science*, 2024.
- [47] Vinnie Wong. Gen AI’s environmental ledger: A closer look at the carbon footprint of ChatGPT. <https://piktochart.com/blog/carbon-footprint-of-chatgpt/>, 2023.
- [48] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga Behram, James Huang, Charles Bai, Michael K. Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin S. Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Kumar Jain, Michael G. Rabbat, and Kim M. Hazelwood. Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems*, 2022.
- [49] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [50] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.